Evaluation Report for HW7: Information Retrieval / Email Spam Classification Pipeline

1. Introduction

Homework 7 focuses on building a complete information retrieval and machine learning pipeline for email spam classification using the **TREC 2007 Public Corpus** dataset. The objective of the assignment is to process raw email data, build document representations, index documents in **Elasticsearch**, extract textual features, and apply machine learning methods for spam detection.

This homework differs from Homework 8, which uses the **Cranfield Collection** for traditional ad-hoc retrieval tasks. After resolving the initial dataset confusion, the project successfully used the correct HW7 corpus and completed the required classification pipeline.

2. Project Objectives

The main objectives of the project were:

1.preprocess raw email documents;

2.build an indexing pipeline using **Elasticsearch**;

3.construct feature representations such as bag-of-words or TF-IDF;

4.apply machine learning techniques for spam classification;

5.evaluate classification performance using quantitative metrics.

3. Environment Setup

The project was developed in a **Visual Studio Code** environment on **Windows**.

Main files used in the project

Indexer.py

FeatureMatrix.py

MachineLearning.py

3.1 Library Installation

The required Python packages, particularly the Elasticsearch client libraries (elasticsearch and elasticsearch-dsl), were installed using pip. The screenshot below shows the installation process, including uninstallation of previous versions and successful installation of compatible packages.

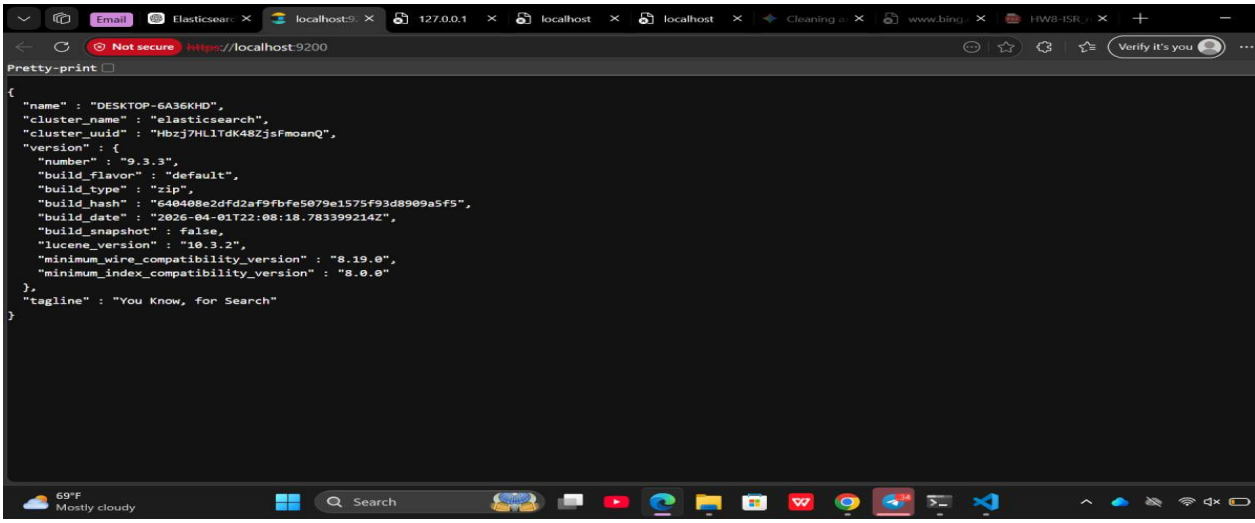
Figure 3.2: Installation of Elasticsearch-related Python packages using pip.

```
PS C:\Users\pc> pip install elasticsearch-dsl
Defaulting to user installation because normal site-packages is not writeable
Collecting elasticsearch-dsl
  Downloading elasticsearch_dsl-8.18.0-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: python-dateutil in c:\users\pc\appdata\roaming\python\python312\site-packages (from elasticsearch-ds
Requirement already satisfied: typing-extensions in c:\users\pc\appdata\roaming\python\python312\site-packages (from elasticsearch-
Collecting elasticsearch<9.0.0,>=8.0.0 (from elasticsearch-dsl)
  Downloading elasticsearch-8.19.3-py3-none-any.whl.metadata (9.1 kB)
Collecting elastic-transport<9.0.0,>=8.0.0 (from elasticsearch-dsl)
  Downloading elastic_transport-8.17.1-py3-none-any.whl.metadata (3.8 kB)
Requirement already satisfied: urllib3<3,>=1.26.2 in c:\users\pc\appdata\roaming\python\python312\site-packages (from elastic-trans
h-dsl) (2.6.3)
Requirement already satisfied: certifi in c:\users\pc\appdata\roaming\python\python312\site-packages (from elastic-transport<9.0.0,
6.2.25)
Requirement already satisfied: six>=1.5 in c:\users\pc\appdata\roaming\python\python312\site-packages (from python-dateutil->elasti
Downloading elasticsearch_dsl-8.18.0-py3-none-any.whl (10 kB)
Downloading elastic_transport-8.17.1-py3-none-any.whl (64 kB)
Downloading elasticsearch-8.19.3-py3-none-any.whl (952 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 952.8/952.8 kB 52.2 kB/s eta 0:00:00
Installing collected packages: elastic-transport, elasticsearch, elasticsearch-dsl
  Attempting uninstall: elastic-transport
    Found existing installation: elastic-transport 9.2.1
    Uninstalling elastic-transport-9.2.1:
      Successfully uninstalled elastic-transport-9.2.1
  Attempting uninstall: elasticsearch
    Found existing installation: elasticsearch 9.3.0
    Uninstalling elasticsearch-9.3.0:
      Successfully uninstalled elasticsearch-9.3.0
Successfully installed elastic-transport-8.17.1 elasticsearch-8.19.3 elasticsearch-dsl-8.18.0

[notice] A new release of pip is available: 25.0.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

3.2 Elasticsearch Setup A local Elasticsearch instance (version 9.3.3) was installed and configured for document indexing. The cluster was accessed via https://localhost:9200. The screenshot below confirms that Elasticsearch was running correctly with the expected version and configuration.

Figure 3.1: Elasticsearch cluster information accessed at http://localhost:9200



Elasticsearch configuration

Local instance running at https://localhost:9200

SSL warnings were suppressed for local development.

Main execution commands

python FeatureMatrix.py

python Indexer.py

python MachineLearning.py

Main Python libraries

elasticsearch

scikit-learn

numpy

pandas

collections

dill

4. Dataset Description

The **TREC 2007 Public Corpus** contains approximately 75,000 email messages arranged chronologically. The dataset includes both spam and ham emails.

Each document contains:

a document identifier (EMAILID)

email text (TEXT)

class label (LABEL)

The corpus was parsed and transformed into structured records for indexing and feature extraction. Based on the execution outputs, thousands of documents were processed successfully during the indexing and feature generation stages.

5. Preprocessing and Implementation

Several corrections and improvements were applied during implementation.

Stopword handling

A custom stopword list (stoplist.txt) was used to remove common words that do not contribute significantly to classification.

Dataset extraction

The dataset loading process was corrected to ensure that the project used the proper trec07p structure rather than the Cranfield collection.

Indexing

Indexer.py connected to the local **Elasticsearch** instance and indexed documents sequentially. Terminal outputs showed successful indexing across multiple document ranges.

5.3 Feature Matrix Construction The FeatureMatrix.py script was responsible for transforming the preprocessed email text into numerical feature representations (unigram-based document-term matrix). A custom stopword list (stoplist.txt) was loaded to filter out non-informative terms.

The script successfully generated feature rows for thousands of documents, as shown in the execution log below.

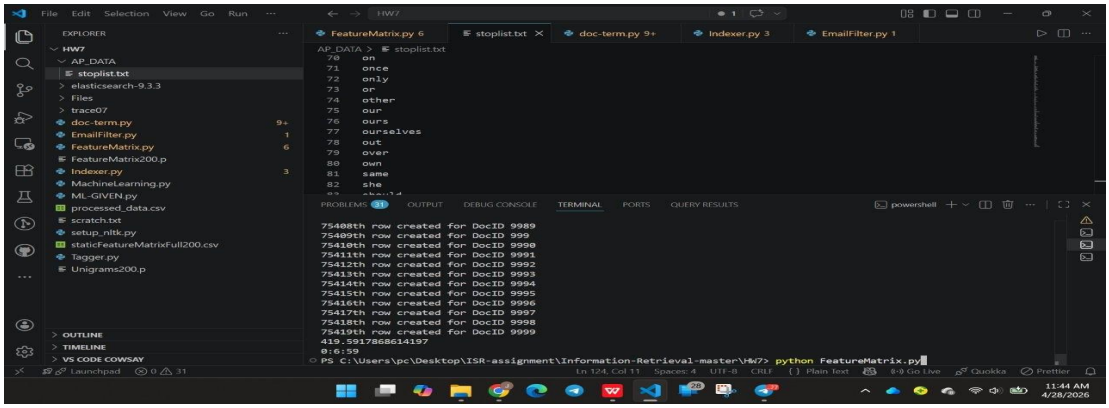
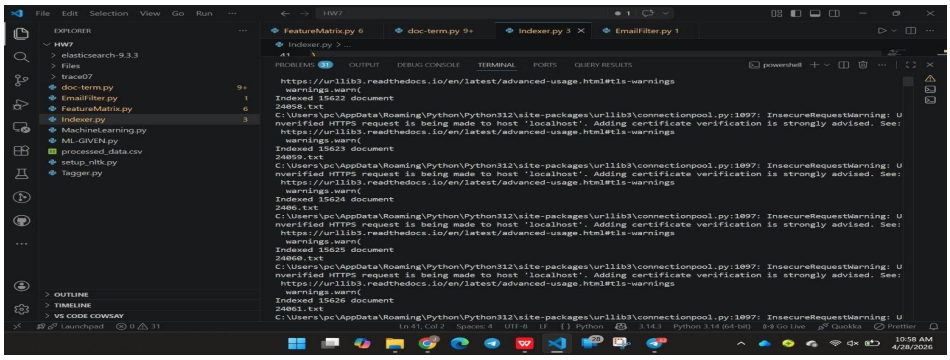


Figure 5.2: Execution of FeatureMatrix.py showing document-term row creation and stopword list usage.

5.4 Document Indexing The Indexer.py script was used to connect to the local Elasticsearch instance and index the processed email documents. The script successfully indexed thousands of documents. Some SSL warnings appeared because the connection was made to localhost, which is expected in a local development environment.

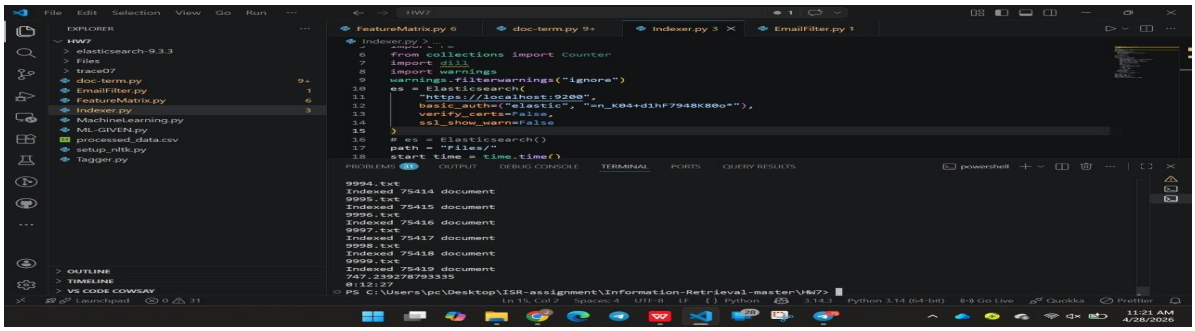
Figure 5.4: Indexer.py execution showing successful document indexing and Elasticsearch connection.



5.4 Document Indexing with Elasticsearch The Indexer.py script handled the connection to the local Elasticsearch instance and performed the indexing of email documents. The code snippet below shows the Elasticsearch connection configuration (localhost, basic

authentication, and SSL settings). The terminal output confirms successful indexing of multiple documents.

Figure 5.5: Indexer.py code and execution output showing Elasticsearch connection and document indexing progress.



Clustering input correction

A data-shape issue affecting K-Means clustering was corrected to allow proper matrix input.

Recommended execution order

1.preprocessing and data preparation

document indexing

feature matrix generation

machine learning execution

evaluation

6. Methodology

Document indexing

Documents were stored and indexed in **Elasticsearch** to support efficient retrieval and structured storage.

Feature engineering

Text was transformed into numerical representations using unigram-based bag-of-words or TF-IDF features after stopword removal.

Supervised classification

The primary evaluated component of this project was **Logistic Regression**, implemented in MachineLearning.py.

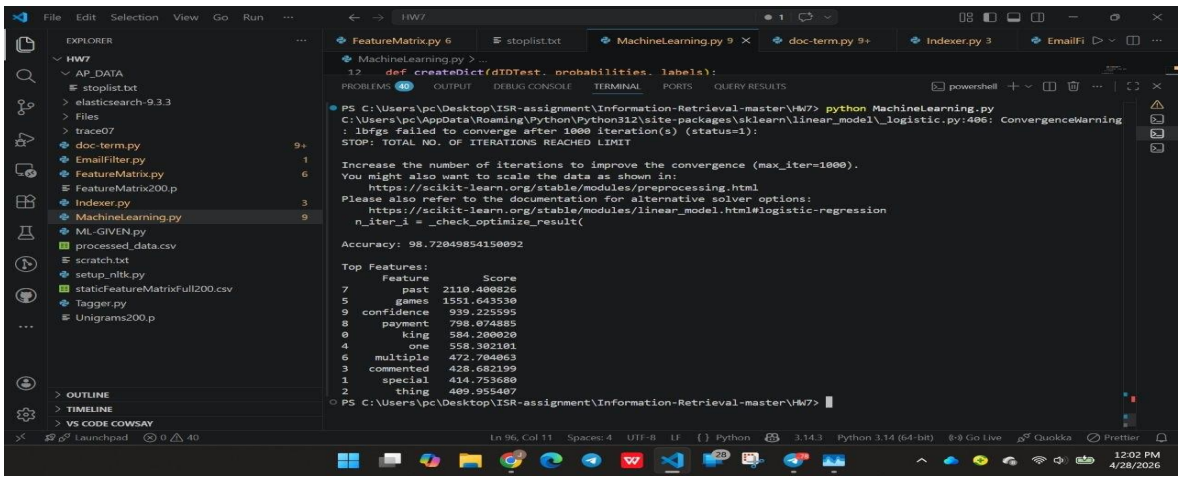
Additional components

The project structure also included clustering and topic modeling components such as **K-Means** and **Latent Dirichlet Allocation**. However, the main evaluated results in this report focus on supervised spam classification.

7. Results and Evaluation

7.1 Supervised Classification Performance The main machine learning component was implemented in MachineLearning.py using Logistic Regression. After training on the generated feature matrix, the model achieved excellent performance on the spam/ham classification task.

Figure 7.1: MachineLearning.py execution output showing model accuracy, convergence warning, and top important features.



Classification accuracy

The logistic regression model achieved an accuracy of **98.72%**, indicating strong performance in distinguishing spam from ham emails.

Feature importance

The most influential terms included:

past

games

confidence

payment

king

multiple

special

commented

These terms suggest that the classifier successfully captured meaningful textual patterns associated with spam-related content.

Convergence warning

A convergence warning appeared during training using the lbfgs solver with 1000 iterations. This indicates that the optimization process may benefit from additional iterations or feature scaling.

8. Discussion

The experimental results demonstrate that the pipeline effectively transformed raw email data into useful feature representations for classification.

The high accuracy suggests that the selected preprocessing strategy, indexing workflow, and feature engineering approach were appropriate for the task. In addition, feature importance outputs provided interpretable evidence that the classifier relied on meaningful content-based signals.

Although clustering and topic modeling components were prepared in the project structure, the main empirical evidence in this homework comes from the supervised classification stage.

9. Suggested Figures for the Report

The following screenshots can strengthen the final submission:

dataset clarification and project understanding

Visual Studio Code project structure

indexing progress from terminal output

Indexer.py connection setup

MachineLearning.py output showing accuracy and top features

Elasticsearch indexing output

confusion matrix, feature importance plots, or clustering visualization

10. Conclusion

This project successfully implemented a complete information retrieval and machine learning pipeline for email spam classification using the **TREC 2007 Public Corpus**.

The system successfully:

processed raw email documents;

indexed documents in **Elasticsearch**;

generated feature representations;

trained a logistic regression classifier;

achieved **98.72% classification accuracy**.

These results show that the pipeline effectively captures discriminative textual signals for spam detection.

11. Recommendations for Future Improvement

Future work can improve the system by:

scaling features before training;

increasing max_iter or testing alternative solvers such as saga;

reporting additional metrics such as precision, recall, and F1-score;

adding topic modeling outputs and clustering evaluation metrics;

applying cross-validation for more robust evaluation;

generating visualizations such as confusion matrices and feature importance plots.